

1. B) Embedding
2. C) \$group
3. c) Projeção
4. c) skip() e limit()
5. b) \$exists

Questão 1:

R: Está sendo utilizada a técnica de Embedding (documentos embutidos ou aninhamento). Os dados do histórico são armazenados como uma lista de subdocumentos diretamente dentro do registro do lead, em vez de isolados em uma coleção externa ligada por chaves. Alta performance de leitura e excelente localidade de dados. Como o histórico está encapsulado no próprio documento, o MongoDB consegue recuperar o lead e todo o seu histórico em uma única operação de IO, eliminando a necessidade de junções (*joins*). Risco de ultrapassar o limite físico de 16 MB por documento caso o histórico cresça indefinidamente. Além disso, arrays que crescem constantemente prejudicam a performance de escrita, pois forçam o banco de dados a realocar o documento em disco repetidamente. Se o limite de 16 MB for excedido, o banco retornará um erro e rejeitará novas inserções.

Questão 2:

R: A abordagem de Referencing (referenciamento por IDs) é mais adequada neste cenário porque promove a normalização e a consistência dos dados, eliminando a redundância e a necessidade de atualizações em cascata. Se os dados de um vendedor fossem embutidos em milhares de documentos de clientes e houvesse a necessidade de alterar uma informação simples (como o telefone do vendedor), o banco teria que executar uma operação de atualização massiva e custosa em toda a base, gerando um alto risco de inconsistência. Centralizando os dados do vendedor em uma coleção própria, qualquer alteração é feita em um único ponto e reflete instantaneamente para todos os clientes associados.

Questão 3:

R: O *Aggregation Framework* soluciona essa demanda através de uma esteira de processamento (*pipeline*) de dados. Os operadores recomendados para cada métrica são:

- Quantidade de leads por origem: Utiliza-se o operador `$group` para agrupar os documentos com base no campo da origem, aplicando o operador acumulador `$sum` (com valor 1) para realizar a contagem dos registros de cada grupo.
- Quantidade de leads por vendedor: Segue o mesmo princípio de agrupamento (`$group` e `$sum`), mapeando o identificador do vendedor.

- Quantidade de negociações concluídas: Primeiramente, aplica-se o estágio `$match` para filtrar e permitir que apenas os documentos com o status "concluído" avancem no pipeline. Em seguida, o operador `$count` (ou um agrupamento com `$sum`) é utilizado para extrair o volume exato.
- Formatação e Organização: Caso seja necessário ordenar os resultados (ex: do mais produtivo para o menos produtivo), utiliza-se o `$sort`. Para mascarar ou selecionar apenas os campos que devem ir para o relatório final, utiliza-se o `$project`.
-

Questão 4:

R: São estruturas de dados especiais e otimizadas que armazenam uma pequena fração do conjunto de dados da coleção de forma ordenada e altamente indexada. Sem índices, o MongoDB é obrigado a realizar uma varredura completa na coleção (COLLSCAN), analisando documento por documento em disco para encontrar o resultado. Ao criar um índice, o banco organiza os valores do campo escolhido em uma estrutura de árvore de busca balanceada (B-Tree). Cada nó dessa árvore contém um ponteiro direto para a localização física do documento correspondente no disco, permitindo buscas cirúrgicas.

Aumento drástico na eficiência e na velocidade das consultas, redução acentuada no consumo de memória RAM e de CPU, e garantia de alta performance mesmo sob grandes volumes de dados.

Questão 5:

R: Para mitigar a duplicidade no nível de banco de dados, podem ser adotadas as seguintes estratégias:

1. Índices Únicos (*Unique Indexes*): É a solução mais robusta. Cria-se um índice único sobre um campo identificador inequívoco (como CPF ou Email). Assim, o MongoDB rejeita nativamente qualquer tentativa de duplicidade, retornando um erro de chave duplicada.
2. Schema Validation: Implementação de regras de validação via *JSON Schema* diretamente na coleção. Isso obriga que todos os dados sigam uma estrutura estrita e padronizada, evitando cadastros inconsistentes.
3. `_id` Customizado: Em vez de permitir que o MongoDB gere um *ObjectId* automático, a aplicação pode definir um identificador de negócio exclusivo (como a combinação de dados ou uma chave única) para preencher o campo `_id`. Como o `_id` é obrigatoriamente único por coleção, o banco impedirá a colisão.

Questão 6:

R:

- Vantagens: Alta flexibilidade no desenvolvimento inicial, visto que o MongoDB é *schema-less*. O banco acomodará sem restrições documentos de naturezas totalmente diferentes (vendedores, leads, etc.) com estruturas distintas na mesma coleção.

- Desvantagens: Desorganização estrutural ("poluição visual") da base à medida que o volume cresce, tornando a manutenção complexa. Além disso, a aplicação de regras de validação (*Schema Validation*) se torna extremamente difícil e confusa dentro de um único ecossistema misto.
- Impactos Futuros: Comprometimento da evolução do código a longo prazo devido à complexidade para filtrar e mapear entidades. Haverá também uma severa degradação da performance global. Como o MongoDB tenta manter os dados mais acessados (os chamados *working sets*) na memória RAM, a mistura de dados fará com que registros secundários ou volumosos expulsem dados críticos da memória, forçando o banco a ler constantemente do disco.

Questão 7:

R: Retornar grandes volumes de dados de uma só vez satura a memória do servidor, sobrecarrega o processamento do banco e consome excessivamente a largura de banda da rede, gerando lentidão. Sob a ótica do usuário (UX), a paginação garante uma interface muito mais fluida, limpa e responsiva.

- Funcionamento dos comandos: * O método `skip(n)` instrui o MongoDB a pular uma quantidade `n` específica de documentos iniciais da consulta.
 - O método `limit(m)` restringe o tamanho do lote de exibição, interrompendo a busca assim que atinge o teto `m` estipulado.
 - Juntos, eles fatiam os dados (ex: para a página 2 com 20 registros por página, usa-se `skip(20).limit(20)`).

Questão 8:

R:

- Vantagens: Agilidade e velocidade no ciclo de desenvolvimento, permitindo o armazenamento de dados dinâmicos e heterogêneos sem a necessidade de migrações complexas de banco de dados. É ideal para cenários cujos atributos variam de registro para registro, como em um catálogo de e-commerce onde as especificações de um "livro" diferem completamente das de um "smartphone".
- Cuidados: A equipe de desenvolvimento deve assumir a responsabilidade pela consistência. É preciso gerenciar o versionamento de documentos (visto que registros antigos e novos coexistirão com estruturas diferentes) e garantir a padronização de nomenclatura de campos na aplicação para evitar divergências (como salvar `cpf` em uns e `c_p_f` em outros). Recomenda-se o uso rigoroso de validações via código ou *Schema Validation* nativo.

Questão 9:

R:

- Desempenho (Leitura vs. Escrita): O *Embedding* oferece maior velocidade em operações de leitura na maioria dos casos, pois recupera dados relacionados de uma só vez. Já o *Referencing* pode ser mais lento para leituras pesadas (exigindo buscas extras), porém entrega excelente

performance de escrita, já que os documentos permanecem menores e as atualizações ocorrem de forma isolada.

- Limites de tamanho: O *Embedding* é severamente limitado pelo teto de 16 MB por documento do MongoDB. O *Referencing* contorna essa barreira dividindo a carga entre múltiplos documentos distintos.
- Manutenção do código: Atualizações no *Embedding* tendem a ser complexas e custosas: alterar um valor replicado (como o nome de uma categoria presente em milhares de produtos embutidos) exige uma varredura massiva e alteração em lote. No *Referencing*, a manutenção é simplificada, pois a informação reside em um único local físico e qualquer modificação reflete de forma centralizada e imediata.

•

Questão 10:

R:

- Para o Sistema (Arquitetura): Permite a utilização de ODMs consolidados como o *Mongoose*, facilitando a inclusão de validações, regras de negócio e *hooks* diretamente na camada de software. Como o Node.js executa JavaScript no backend e o MongoDB adota uma sintaxe nativa baseada em JavaScript (manipulando dados em formato BSON), elimina-se a necessidade de camadas pesadas de mapeamento ou tradução de objetos, simplificando radicalmente a arquitetura.
- Para o Usuário Final: Ambas as tecnologias possuem excelente capacidade de escalabilidade sob alto estresse, o que impede quedas ou lentidão no sistema durante picos de acessos. Além disso, a natureza assíncrona e orientada a eventos do Node facilita a criação de recursos em tempo real (como chats e notificações push), atualizando os dados provenientes do banco direto na tela do usuário sem recarregar a página.

•

Questão 11:

R: Considerando um modelo onde as negociações referenciam o ID do vendedor, a estratégia ideal consiste em um pipeline de agregação associado a um índice focado.

- Pipeline de Agregação (3 etapas):
 1. `$match`: Filtra imediatamente os documentos da coleção trazendo apenas as negociações que possuem o status correspondente a "concluído".
 2. `$group`: Agrupa os registros pelo identificador do vendedor (`_id: "$vendedor"`) e contabiliza o volume total acumulado usando o operador `$sum`.
 3. `$sort`: Ordena os resultados obtidos de forma decrescente para expor o ranking dos maiores para os menores produtores.
- Otimização: Cria-se um índice composto ou parcial (utilizando `createIndex`) cobrindo os campos de status e do vendedor para que o pipeline execute instantaneamente direto na memória.

- Justificativa: Essa estratégia garante uma altíssima performance, pois o índice restringe a busca aos dados estritamente necessários. Além disso, o pipeline confere total flexibilidade para evoluções futuras, permitindo agregar novas métricas com facilidade (como calcular o faturamento total além do volume de vendas).

•

Questão 12:

R: Para garantir que um sistema com milhões de registros mantenha-se escalável e eficiente, duas decisões são fundamentais:

1. Adoção de Referencing para relações extensas: Em relações do tipo "um-para-muitos" que tendem a crescer sem controle (ex: milhões de registros associados), o referenciamento é obrigatório para mitigar o estouro do limite físico de 16 MB por documento imposto pelo *Embedding*.
2. Uso estratégico de Índices Parciais: Índices consomem muita memória e o MongoDB foi projetado para operar com eles preferencialmente alocados na RAM. Para otimizar esse espaço, deve-se criar índices parciais (que indexam apenas documentos que atendem a um critério específico, como campos que aparecem em apenas 5% da base). Isso economiza memória RAM preciosa, evitando que o banco precise recorrer a buscas lentas diretamente no disco rígido.

Questão 13:

R: Não, o uso de embedding não é adequado para esta situação. O modelo de documentos embutidos deve ser priorizado apenas para dados correlatos com crescimento previsível e controlado. Como o histórico de interações com o cliente é um fluxo contínuo e ilimitado, a estrutura enfrentará sérios gargalos no longo prazo:

- Risco iminente de estourar o teto de 16 MB do documento.
- Complexidade excessiva e alto custo de processamento no *Aggregation Framework* para realizar filtros simples ou paginação de dados históricos (operações como `skip` e `limit` dentro de arrays internos consomem muita CPU).
- Degradação de IO por realocação: Sempre que um array embutido cresce e o documento excede o espaço previamente reservado em disco, o MongoDB é forçado a reescrever o documento inteiro em um novo bloco físico. Multiplicando isso por milhares de clientes ativos, o sistema sofrerá com severa fragmentação de disco e alto overhead de leitura/escrita, tornando as operações lentas.

Questão 14:

R: A escolha entre o MongoDB e um banco de dados relacional tradicional envolve ponderar diferenças fundamentais em quatro pilares estruturais:

- **Estrutura dos Dados:** O SQL tradicional adota uma organização extremamente estrita e rígida, estruturada em tabelas compostas por linhas e colunas bem definidas, dependendo de chaves estrangeiras para conectar as informações. Em

contrapartida, o MongoDB utiliza o formato BSON (JSON binário), o que permite uma composição muito mais rica com estruturas aninhadas, arrays e objetos internos consolidados em um único documento.

- **Flexibilidade e Evolução do Sistema:** Nos sistemas SQL, qualquer alteração estrutural exige alto planejamento e burocracia técnica, demandando a execução de scripts de migração (*migrations*) para alterar as tabelas, o que traz riscos de travamento e erros na base de dados. Já o MongoDB opera em tempo real de forma *schema-less*; caso surja a necessidade de um novo campo, a aplicação simplesmente envia o documento com a nova propriedade e o banco a grava imediatamente, sem necessidade de paradas ou rotinas complexas.
- **Escalabilidade:** O modelo SQL foi desenhado primordialmente para uma escalabilidade **vertical**, o que significa que, para suportar um volume maior de dados ou acessos, a solução padrão é investir em hardware, transferindo o banco para uma máquina mais potente e dispendiosa. O MongoDB, por outro lado, destaca-se pela escalabilidade **horizontal**, permitindo distribuir a carga e os dados nativamente de forma fragmentada entre múltiplos servidores ou instâncias em nuvem.

Questão 15:

R; A escolha do MongoDB como o principal mecanismo de banco de dados para o ecossistema corporativo fundamenta-se em três pilares técnicos indispensáveis para a sustentabilidade do negócio:

1. Escalabilidade Horizontal Nativa (Sharding): Frente ao crescimento volumétrico projetado, o MongoDB assegura uma infraestrutura escalável através da distribuição horizontal da carga por múltiplos servidores de baixo custo. Isso contorna as limitações físicas e os custos proibitivos atrelados à escalabilidade puramente vertical dos bancos de dados relacionais.
2. Flexibilidade de Esquema e Sinergia com Node.js: A arquitetura *schema-less* confere agilidade ao time, eliminando janelas de manutenção e indisponibilidades causadas por migrações rígidas de tabelas. Ademais, o armazenamento nativo em formato BSON cria um fluxo de dados direto e sem atrito com o ecossistema Node.js, mitigando a necessidade de pesadas e lentas camadas de tradução de objetos.
3. Alta Performance em Consultas e Relatórios Analíticos: O arranjo baseado em índices robustos acoplado ao poderoso *Aggregation Framework* habilita respostas rápidas (na ordem de milissegundos) para extração de relatórios complexos em tempo real. Isso viabiliza uma tomada de decisão ágil pela diretoria, garantindo um ambiente veloz, estável e economicamente viável para suportar a expansão da empresa no longo prazo.